

BENCHSCOUT: Accelerating System Benchmarking with a Robust Machine Learning Pipeline

Anonymous Author(s)

Abstract

Benchmark suites are vital for evaluating computing platforms, but their comprehensive, all-or-nothing execution model incurs prohibitive time and resource costs. While replacing physical execution with machine learning (ML) predictions is a tempting alternative, applying naïve ML directly to system benchmarking fails in practice due to highly dynamic runtime contention, inflexible run-to-completion workloads, and inherently imperfect production telemetry. We present BENCHSCOUT, a systems toolchain that *accelerates* repeated execution of standard suites by keeping sparse real measurements in the loop: it selects a small informative subset, observes each component through a bounded probe, and reconstructs the official suite profile. BENCHSCOUT targets iterative tuning and regression workflows that need the same suite indices at lower cost; it is not a substitute for certified run-to-completion submissions where auditability requires verbatim protocol execution. Inspired by Mixture-of-Experts routing, BENCHSCOUT collects live system context, runs time-bounded probes on Top-K components, and reconstructs the remaining scores. Execution anomalies and missing telemetry are encoded as features rather than hard failures. Across heterogeneous hosts, BENCHSCOUT recovers aggregate suite scores within sub-percent error while reducing wall-clock time by two orders of magnitude.

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper;** *Generate the Correct Terms for Your Paper;* Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper;

Keywords: System Benchmarking, Benchmark Suites, Performance Evaluation, Machine Learning, Mixture of Experts, Sparse Measurement, Time-bounded Probing

ACM Reference Format:

Anonymous Author(s). 2018. BENCHSCOUT: Accelerating System Benchmarking with a Robust Machine Learning Pipeline. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Benchmark suites [?] characterize modern computing platforms by running standardized workloads across processors, memory, storage, system software, and accelerators to produce a repeatable performance profile beyond any single measurement. These suites are widely used in performance-critical infrastructure workflows, including cloud instance comparison [?], monitoring [?], and performance autotuning [?]. However, the same breadth that makes benchmark suites useful also makes them costly to execute in full. As Figure 1 illustrates, benchmark evaluation can easily dominate the overall time budget of an auto-tuning task (Figure 1a) and incur substantial wall-clock overhead across different machine configurations (Figure 1b). Comprehensive suites often contain many heterogeneous workloads, each consuming wall-clock time and contending for CPU cores, memory bandwidth, storage bandwidth, or accelerator resources. While this cost may be acceptable for occasional offline evaluation, it becomes difficult to amortize when benchmarking must be repeated across many machines, instance types, software revisions, configurations, or time intervals. In deployed or shared environments, full-suite execution can also perturb the system being measured by competing for resources, interfering with co-located workloads, or changing thermal and frequency states. Moreover, conventional suites largely follow an all-or-nothing execution model: they run a fixed set of workloads and produce meaningful results only after the full run completes.

In the middle ground, execution-free predictors attempt to bypass benchmark runs by estimating performance from static host configurations, public specifications, or offline training data [?].

Because the system itself cannot magically compress the execution time of standard workloads, various approaches have attempted to mitigate this overhead. “Static-subset” approaches manually cherry-pick a fixed group of subtests, but they face a high barrier to generalization: a subset chosen for a CPU-bound instance falls short on an I/O-heavy machine [?]. However, without executing the target workloads on the current system, such predictors cannot directly

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

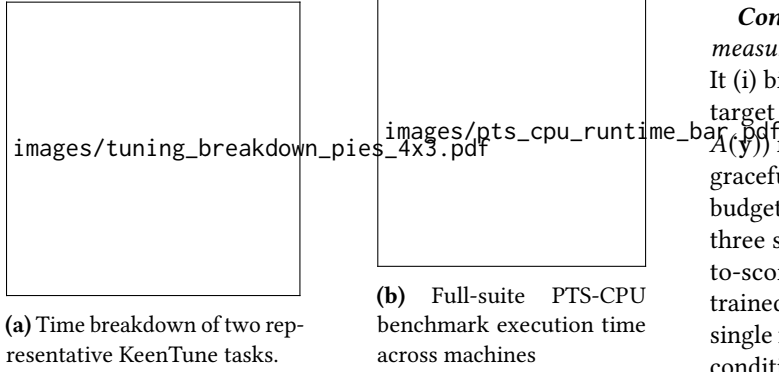


Figure 1. Execution-time cost of benchmark-driven tuning and full-suite benchmark evaluation.

observe transient runtime effects. As a result, they can be unreliable under distribution shifts common in production environments. When a trustworthy performance profile is required, practitioners therefore still fall back to full-suite run-to-completion, preserving measurement fidelity at the cost of long delays and resource interference in shared environments.

We take a different approach: rather than eliminating benchmark execution, BENCHSCOUT keeps real measurements in the loop but makes them sparse and adaptive. It reframes full-suite evaluation as conditional subset selection plus sparse reconstruction: under the current system state, BENCHSCOUT observes only a small informative fraction of the suite and estimates the remaining official component scores and aggregate index.

Problem scope and credibility. BENCHSCOUT addresses two related but distinct needs. **(i) Accelerated formal benchmarking:** autotuning, instance comparison, and regression testing repeatedly invoke standard suites and require estimates of the *same* published indices ($Y = A(y)$) at far lower wall-clock and interference cost than run-to-completion execution. **(ii) Fast performance portraits:** monitoring and capacity triage need a timely snapshot of how a host would likely score today, accepting approximation when a 30-minute full run is impractical. These modes impose different credibility bars. Accelerated formal benchmarking demands traceable sparse execution, per-host calibration against historical full runs, and explicit relative-error reporting against the full-run reference on the same host. Fast portraits tolerate higher approximation error but still require bounded probes, degradation-safe telemetry, and a fixed wall-clock budget that does not scale with suite cardinality. BENCHSCOUT does *not* claim to replace audited benchmark submissions; when certification matters—as with industrial SPEC submissions—its output is a preview and a run-to-completion execution remains the authoritative reference.

Contributions. Systems: BENCHSCOUT is an end-to-end *measurement pipeline*, not an execution-free score regressor. It (i) binds every prediction to bounded real probes on the target host, (ii) preserves official suite semantics ($y, Y = A(y)$) rather than inventing surrogate metrics, (iii) degrades gracefully when PMU/eBPF are blocked, and (iv) reports fixed budgets (K, T_{probe}) suitable for shared production hosts. **ML:** three supervised modules—conditional Top-K router, prefix-to-score probe estimator, and masked reconstructor—are trained per host from historical full runs and composed in a single forward pass (Section 3.1). The router ranks experts by conditional informativeness; probe estimators map bounded telemetry to official component scores; the reconstructor imputes unobserved experts and the suite index from sparse tuples. ML selects *which* experts to measure and *how long* to probe; the OS executes the workloads and collects telemetry. We evaluate accelerated formal benchmarking (primary) and fast portraits (secondary) on five heterogeneous Linux hosts across UnixBench and PTS (Section 4).

2 Background and Motivation

2.1 Benchmark Suites

Benchmark suites provide reproducible and quantifiable measurements for comparing system performance across machines, configurations, and software versions. Rather than relying on a single workload, a suite decomposes system performance into multiple components, each targeting a distinct workload pattern across computation, memory, storage, networking, process control, and inter-process communication. We denote a suite as

$$\mathcal{B} = \{b_1, b_2, \dots, b_N\}, \quad (1)$$

where b_i is the i -th component.

For a system state s , the full result of component b_i is

$$y_i = f_i(s, T_i, \epsilon_i), \quad (2)$$

where T_i is the execution budget required by the original benchmark protocol and ϵ_i denotes measurement noise. A full-suite run produces

$$y = [y_1, y_2, \dots, y_N], \quad (3)$$

and, when defined by the benchmark framework, an aggregate score

$$Y = A(y). \quad (4)$$

Section 3.5 instantiates $A(\cdot)$ for UnixBench and PTS.

The cost of full-suite benchmarking comes from two sources: the number of components and the duration of each component. Formally,

$$C_{\text{full}} = \sum_{i=1}^N C_i(T_i). \quad (5)$$

This cost can be substantial for suites with many components or long measurement intervals. Practitioners invoke

suites for two reasons with different credibility needs (Section 1): repeated *accelerated formal* evaluation that must approximate the same Y , and *fast portraits* when a full run is infeasible. However, not all observations are equally informative: components stress overlapping subsystems (Section 2.2), and prefix executions often expose the same rate trend as run-to-completion. Our goal is therefore to estimate (y, Y) from sparse, time-bounded probes while keeping real measurements in the loop: every reported score is either directly probed on the target host or reconstructed from probe-imputed partial observations under the official aggregator $A(\cdot)$.

2.2 Motivation

Insight 1: Sub-benchmarks Exhibit High Subsystem Redundancy. Although comprehensive suites contain dozens of subtests, many workloads stress overlapping hardware subsystems. Our profiling reveals that seemingly diverse UnixBench tasks—integer computation (*dhry2reg*), process creation (*spawn*), and inter-process communication (*pipe*)—often bottleneck on the same OS scheduling overheads or memory-hierarchy mechanisms. Because these subtests are merely different software projections of the same underlying hardware capacity, executing all of them yields diminishing returns in information gain. This physical redundancy indicates that a carefully routed, minimal subset of experts is theoretically sufficient to expose the current system’s overall performance limits.

To make this redundancy explicit and quantifiable, we pool $n=25$ complete single-copy UnixBench runs (*perl Run -c 1*) collected across our five evaluation hosts (2 U/8 GiB through 32 U/128 GiB; Table 2), yielding five independent full-suite repetitions per machine. For each run we extract the official per-subtest index score for all $N=12$ components and compute pairwise association matrices. Figure 2 reports both Pearson (linear) and Spearman (rank) correlations side-by-side; Spearman is less sensitive to outliers when suite indices span more than a $2\times$ range across hosts (roughly 1590–3190 on our testbeds). The heatmaps show dense CPU clusters with $r \geq 0.99$ among *dhry2reg*, *whetstone-double*, *pipe*, and *syscall*, and equally tight filesystem clusters among *fstime*, *fsbuffer*, and *fsdisk*. Cross-cluster coupling is weaker: *spawn* versus filesystem tests yields $r \approx 0.2$ – 0.3 . This structure justifies selecting $K \ll N$ diverse experts and reconstructing correlated components rather than executing all N subtests.

Insight 2: Performance Metrics Exhibit Early Convergence. When full-suite execution is unavoidable, traditional methodologies mandate a rigid run-to-completion model. However, many benchmark components are fundamentally iterative rate tests: the Dhrystone and Whetstone components in UnixBench [?] continuously execute integer and floating-point kernels over a prolonged loop and report an accumulated operation rate in operations per second. On our

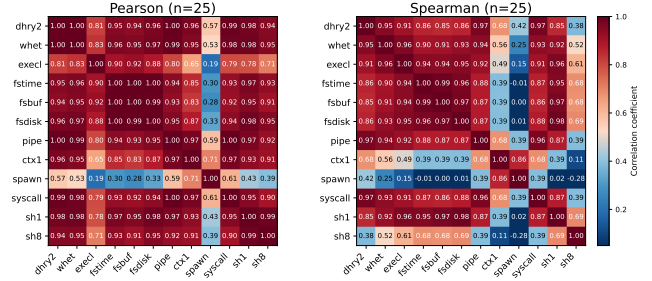


Figure 2. Pairwise UnixBench subtest score correlation on five heterogeneous hosts ($n=25$ single-copy runs). Left: Pearson r ; right: Spearman ρ . Dark red marks redundant pairs; lighter cells indicate weaker coupling for complementary Top-K selection.

testbeds, the prefix rate sampled at 0.5 s intervals typically stabilizes within the first 4–8 s of execution, while official UnixBench components may run for minutes. Continuing the workload to its official timeout primarily reduces statistical variance rather than altering the fundamental performance metric. This gap between convergence time and protocol time motivates vertical cost reduction via aggressive, time-bounded probing (Section 3.4).

Insight 3: Production Telemetry is Inherently Fragile. Short probes and eBPF/PMU tracing routinely encounter timeouts, permission denials, and missing counters on shared cloud hosts. Hypervisors may block perf event groups; tenant isolation may disable bpftrace; and heavy neighbor load can prevent a 4 s micro-probe from reaching a steady rate. A deployable toolchain must record these conditions—timeout flags, tracer-availability bits, elapsed wall time—as first-class features and continue inference rather than discard runs as statistical outliers. This requirement shapes both feature collection (Section 3.2) and probe training (Section 3.4).

3 Methods

3.1 Overview

BENCHSCOUT approximates a complete benchmark run by observing only a small informative subset (Figure 3; scope in Section 1). Given the current system state, it collects context x , routes Top- K subtests, runs fixed-duration probes on the selected components, and reconstructs the full official profile (Sections 3.3–3.5). The pipeline is *single-pass*: budgets (K , probe duration) are fixed at deployment; there is no adaptive re-probing loop.

Offline training and online protocol. **Offline (per host h):** collect M complete runs $\{(x_i, y_i, Y_i)\}_{i=1}^M$ and train (1) router f_θ from (x_i, y_i) , (2) probe estimators f_{e_j} from $(p_{i,j}, y_{i,j})$, and (3) reconstructor g_ϕ from masked partial vectors and full labels. **Online:** BENCHSCOUT executes four deterministic stages on the target host: (1) **Profile**—collect x with 3 s

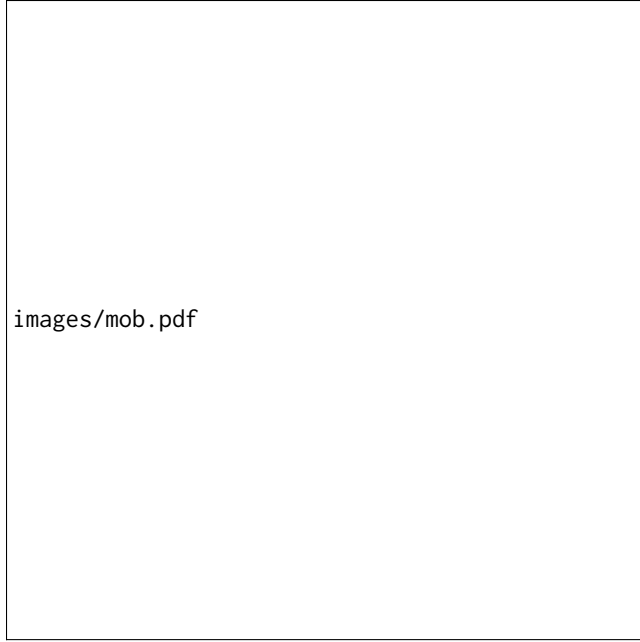


Figure 3. Overview of the BENCHSCOUT architecture. (1) **Host OS**: collector aggregates static and dynamic metrics into $\mathbf{x}$. (2) **Router**: scores experts and selects Top- K . (3) **Probing**: category micro-workloads (or real subtests) run under a fixed budget and emit $\mathbf{p}$. (4) **Reconstructor**: multi-output regressor predicts $\hat{\mathbf{y}}$ and $\hat{Y}$; an optional uncertainty head estimates per-target σ (Section 3.5) but is *not* used in the default single-pass pipeline.

warmup and 0.5 s /proc sampling (perf PMU when permitted, otherwise /proc-only proxies); (2) **Route**—score experts with f_θ and select $\mathcal{S}_K(\mathbf{x})$; (3) **Probe**—for each $e_j \in \mathcal{S}_K$, run one micro-probe (default $T_{\text{probe}}=4$ s) and predict $\hat{y}_j = f_{e_j}(\mathbf{p}_j)$; (4) **Reconstruct**—assemble $Z = [\mathbf{x}, m_j, \hat{y}_j, \tau_j]$ and output $\hat{\mathbf{y}}, \hat{Y} = g_\phi(Z)$ with $\hat{Y} = A(\hat{\mathbf{y}})$. Default wall time is $T_x + K \cdot T_{\text{probe}}$ (≈ 15 s for $K=3$) versus C_{full} (Equation 5); probe error on observed components is absorbed by g_ϕ when imputing unobserved scores. Unlike execution-free predictors [? ?], BENCHSCOUT binds every prediction to bounded on-host probes, live telemetry with explicit degradation paths, official aggregation $A(\cdot)$, and per-host training.

3.2 Feature Collection and Engineering

For run i , the pre-run system context is

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{static}}, \mathbf{x}_i^{\text{dynamic}}], \quad (6)$$

where $\mathbf{x}_i^{\text{static}}$ covers stable descriptors (CPU topology, memory, OS/compiler versions) and $\mathbf{x}_i^{\text{dynamic}}$ captures transient pre-run state (load, runnable tasks, perf//proc counters). During probing, each component b_j additionally yields a prefix vector $\mathbf{p}_{i,j}$ with wall time, context-switch rates, and workload identifiers (Table 1).

Feature Processing and Fallbacks. BENCHSCOUT uses domain-aware scaling rather than global MinMax/z-score normalization: static capacities keep physical magnitudes; dynamic counters become ratios or per-second rates; categorical and failure indicators are one-hot encoded; missing values are zero-filled for deterministic inference. When perf, eBPF, or GPU telemetry is unavailable, the collector records degradation flags and zero-masks the affected dimensions instead of aborting, preserving deployability on restricted cloud kernels.

3.3 Router: Conditional Expert Selection

Because sub-benchmark outputs are correlated projections of the same system state (Section 2.2), a small diverse subset can suffice for full-suite reconstruction; the optimal subset depends on the current context $\mathbf{x}$. BENCHSCOUT therefore treats benchmark acceleration as conditional expert selection: given $\mathbf{x}$, the router picks sub-benchmarks whose measurements are most useful for reconstruction.

Expert Routing. Each executable sub-benchmark is a measurement expert in a fixed pool:

$$\mathcal{E} = \{e_1, e_2, \dots, e_n\}. \quad (7)$$

Given $\mathbf{x}$, scoring model f_θ assigns each expert a relevance score:

$$s_i = f_\theta(\mathbf{x}, e_i). \quad (8)$$

The scores are normalized into selection probabilities:

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^n \exp(s_j)}. \quad (9)$$

The router then executes the Top- K experts:

$$\mathcal{S}_K(\mathbf{x}) = \text{TopK}_{e_i \in \mathcal{E}} p_i. \quad (10)$$

The resulting partial observation vector is passed to the reconstructor to estimate the full-suite profile.

Training Objective. For each run i , all experts share context $\mathbf{x}_i$ and receive label $y_{i,j}$ from $\mathbf{y}_i$. LightGBM routers minimize pairwise ranking loss; neural routers minimize cross-entropy to normalized labels $\{y_{i,j} / \sum_k y_{i,k}\}$. The router ranks conditionally informative experts; suite recovery is delegated to the reconstructor. Budget K trades accuracy for cost: smaller K saves time but leaves fewer observations for reconstruction.

3.4 Time-Bounded Probing

Beyond horizontal subset selection, BENCHSCOUT reduces per-expert cost via time-bounded probing: a fixed wall-clock window replaces run-to-completion execution, and a supervised estimator maps prefix telemetry to the official full-run score (Section 2.2).

Table 1. Purpose-oriented feature groups organized by collection stage. Pre-run signals form the system context $\mathbf{x}_i$, while in-run probe signals form the probe vector $\mathbf{p}_{i,j}$.

Stage	Signal type	Purpose	Representative signals
Pre-run	Static host profile	Identify stable host configuration.	CPU topology, NUMA layout, memory size, compiler versions
	System pressure	Capture transient pre-run load.	CPU/GPU utilization, iowait, load average, runnable tasks
	PMU / perf	Characterize microarchitectural behavior.	IPC, cycles/s, instructions/s, cache/branch miss ratios
In-run	Probe progress	Record observed benchmark progress.	Configured probing duration, elapsed wall-clock time
	eBPF-based tracing	Observe prefix-level kernel activity.	Scheduling-switch rate, syscall-entry rate, eBPF availability
	Resource usage	Summarize prefix resource usage.	User/system/idle CPU ratio, minor faults, major faults
	Workload identity	Provide workload semantic context.	CPU, memory, I/O, thread, syscall, GPU category
	Probe flags	Mark probe mode and failures.	Micro workload flag, real-component flag, timeout flag

Dual-Mode Probing. **Micro-probes** map expert e_j to a coarse category (CPU, memory, I/O, syscall, thread, GPU) and run a short synthetic stimulus (e.g., sqrt loop for *dhry2reg*, temp-file I/O for *fstime*) while collecting eBPF/proc telemetry. **Real-probes** invoke the original subtest under a hard timeout. Both emit bounded telemetry $\mathbf{p}_{i,j}$ and predicted score $\hat{y}_{i,j}$; supervision always targets the official full-run label $y_{i,j}$. Formally, for run i and expert e_j :

$$\hat{y}_{i,j} = f_{e_j}(\mathbf{p}_{i,j}^{\text{micro}}), \quad (11)$$

or, in the shared-estimator setting,

$$\hat{y}_{i,j} = f(\mathbf{p}_{i,j}^{\text{micro}}, \text{id}(e_j)), \quad (12)$$

where $\text{id}(e_j)$ one-hot-encodes expert identity. Production telemetry is often incomplete; BENCHSCOUT encodes anomalies as features rather than discarding runs:

- **Truncation:** timeout events set *real_timed_out* and record elapsed wall time.
- **eBPF fallback:** when tracing is unavailable, *ebpf_available*=0 and eBPF dimensions are zero-masked.
- **Rate normalization:** monotonic counters (*sched_switch*, *syscall_enter*) convert to per-second rates.

Training objective. The estimator minimizes $\mathcal{L}_{\text{probe}} = \sum_{i,j} (\hat{y}_{i,j} - y_{i,j})^2$ in official label space (UnixBench indices; $\log(1+y)$ -transformed PTS primaries), yielding fixed T_{probe} at inference.

3.5 Reconstructor: Full-Suite Result Recovery

Given system context and sparse probe-imputed partial scores, the reconstructor is a multi-output regressor that predicts all unexecuted component scores and the aggregate suite index.

Input Representation. Each expert e_i is a 3-tuple $\mathbf{z}_i = (m_i, v_i, \tau_i)$ with mask m_i , score v_i , and probe wall time τ_i

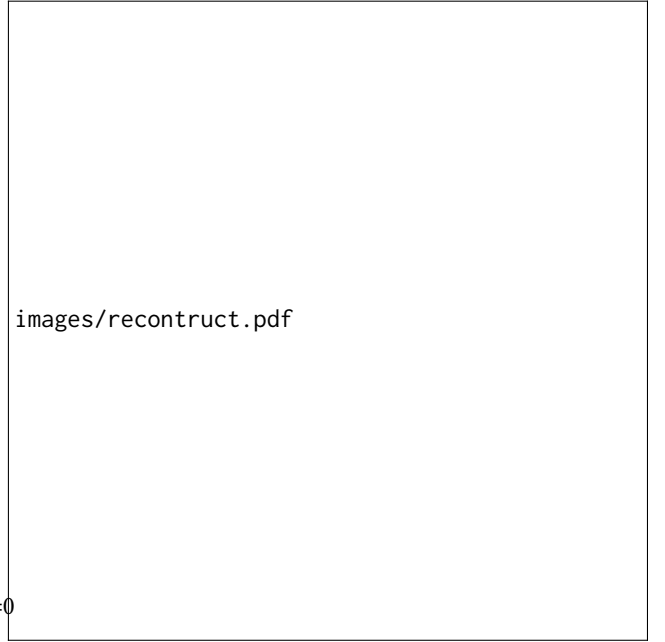


Figure 4. Reconstructor training with random-masked full runs. Historical full-run results are randomly masked to generate augmented sparse samples. The reconstructor takes the masked scores and binary mask as input, predicts the full-run result, and is optimized using the original full-run result as the supervision target.

(zero-filled if unexecuted). The reconstructor input is

$$\mathbf{X} = [\mathbf{x}, m_1, v_1, \tau_1, \dots, m_n, v_n, \tau_n] \quad (13)$$

Target $\mathbf{Y} = [r_1, \dots, r_n, S]$ includes all component scores and suite index S ; tree models use independent regressors per dimension.

Table 2. Evaluation testbeds. Five Linux hosts with heterogeneous CPU and memory capacity. CPU models are taken from `/proc/cpuinfo` at collection time. PTS-GPU runs only on H1.

Host	vCPU (CPU model)	RAM	GPU
H1	32(Intel Core i9-14900K)	128GB	NVIDIA RTX 3090 Ti GPU
H2	2(Intel Xeon 6982P-C)	8GB	—
H3	4(Intel Xeon 6982P-C)	8GB	—
H4	4(Intel Xeon 6982P-C)	16GB	—
H5	8(Intel Xeon Platinum)	8GB	—

Training and aggregation. Historical full runs are augmented with random masks (Figure 4) and supervised with $\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 + (\hat{Y} - Y)^2$. Labels follow each suite’s official semantics (Section 3.5): UnixBench uses per-subtest *Index* scores and the published *System Benchmarks Index Score*; PTS profiles use primary throughput/latency values aggregated via log-mean:

$$A_{\text{logmean}}(\mathbf{v}) = \exp\left(\frac{1}{N} \sum_{j=1}^N \log(1 + v_j)\right) - 1. \quad (14)$$

Probe and reconstructor inputs for PTS use $\log(1 + y)$ scaling; suite supervision always uses the official Y . When all components are probe-predicted without reconstruction, UnixBench suite scores use the geometric-mean-over-indices rule $A_{\text{geommean}}(\hat{\mathbf{y}}) = \exp(\text{mean}_j(\log(1 + \hat{y}_j))) - 1$.

Optional uncertainty head. An optional uncertainty head (Figure 3) predicts per-target σ_k from training residuals (tree models) or Gaussian NLL (MLP twin-head outputs), exported with v2 model bundles but *not* used in the evaluated single-pass Hybrid pipeline (fixed K , no adaptive re-probing).

4 Evaluation

4.1 Experimental Setup

Hardware environment. We deploy BenchScout on five heterogeneous Linux hosts (Table 2). H1 (32 vCPUs, Intel Core i9-14900K, 128 GiB RAM, NVIDIA RTX 3090 Ti GPU) is the development server and sole PTS-GPU host; H2–H5 are smaller CPU-only cloud instances (Intel Xeon 6982P-C or Xeon Platinum) for UnixBench and PTS-CPU. All machines share Ubuntu 22.04.5 LTS and GCC 11.5.0; H1 additionally uses NVIDIA driver 580.159.03, PyTorch 2.12.0+cu130, and TensorFlow 2.21.0. We label hosts by a compact CPU×RAM shorthand used throughout this section.

Software environment. Each host runs a Linux kernel with standard perf and optional bpftrace support. System feature collection ($\mathbf{x}$) uses a 3 s warmup, 0.5 s /proc sampling, and a 64 MiB in-process warmup working set unless noted otherwise. When perf PMU counters are blocked, the collector falls back to /proc-based proxies without aborting

the pipeline. On H1, NVIDIA driver and OpenCL inventory are collected when available; on CPU-only hosts (H2–H5), GPU dimensions are zero-masked at inference.

Benchmark Suites. We evaluate on three benchmark suites that stress complementary subsystems. UnixBench and PTS-CPU are run on all five hosts; PTS-GPU is restricted to H1 because the other four machines lack discrete GPUs.

- **UnixBench (UB).** A classic system-level suite with 12 single-copy subtests ($N = 12$): *dhry2reg*, *whetstone-double*, *execl*, *fstime*, *fsbuffer*, *fsdisk*, *pipe*, *context1*, *spawn*, *syscall*, *shell1*, and *shell8*. The suite spans CPU kernels, filesystem I/O, and OS/interaction workloads. We run UnixBench with a single parallel copy (*perl Run -c 1*), matching the BenchScout collection protocol. The aggregate metric is the *System Benchmarks Index Score*.
- **Phoronix Test Suite — CPU (PTS-CPU).** The standard CPU suite with approximately 13 profiles spanning compilation (*build-linux-kernel*), cryptography, compression, and numerical kernels. Each profile reports a primary throughput or latency metric; suite aggregation follows log-mean over profiles, consistent with our reconstructor training target.
- **Phoronix Test Suite — GPU (PTS-GPU).** The NVIDIA GPU compute suite, executed only on H1, which has a suitable NVIDIA GPU and installed PTS profiles. This suite complements CPU-centric evaluation with OpenCL/GPU compute workloads and is omitted on H2–H5.

On each host, we collect multiple complete benchmark sessions (typically 3–5 rounds per session) for every suite applicable to that host.

Why UnixBench and PTS, not SPEC CPU2017. SPEC CPU2017 is the industrial processor reference, but our target workflows—cloud auto-tuning, regression testing, and operator monitoring—need open, scriptable suites that exercise CPU, I/O, OS, and GPU subsystems without license-managed publication. We evaluate on UnixBench and PTS accordingly; certified SPEC submission lies outside our scope (Section 1).

Baselines. We compare BENCHSCOUT against three baselines. The *full-run* baseline executes the entire suite and provides the reference result with no runtime reduction. Wang et al. [?] train a deep neural network to predict processor benchmark scores from public CPU specifications and historical benchmark tables. Touse et al. [?] study supervised regression for predicting SPEC CPU2017 results from hardware and software configurations—representative of industrial-grade, execution-free SPEC prediction pipelines. We include Touse et al. to test whether SPEC-trained models transfer to UnixBench and PTS on the same hosts. Both execution-free baselines use leave-one-run-out cross-validation; wall time is $\mathbf{x}$ collection only (≈ 3.5 s). Unlike these prediction-only baselines, BENCHSCOUT selectively runs a small subset of

experts and reconstructs the full-suite result from bounded on-host probes. All methods share the same train/test splits whenever the required input features are available.

Evaluation organization. The remainder of Section 4 is organized around three execution paths. **(1) Hybrid pipeline (BENCHSCOUT main method):** collect $\mathbf{x}$, router Top- K , micro-probe selected components, reconstruct the suite score (router $\rightarrow$ probe $\rightarrow$ reconstruct). We first report five-host end-to-end effectiveness (Section 4.2), followed by a runtime overhead analysis using UnixBench resource waveforms (Section 4.3). **(2) Router only ablation:** replace probe predictions with *measured* partial sub-benchmark scores on the router-selected subset, then reconstruct (Section 4.4), with supplementary offline studies on Top- K and system-feature ablation (Sections 4.6, 4.7). **(3) Probe only ablation:** probe every component without router selection or reconstruction (Section 4.5), followed by telemetry-degradation and probe-duration sweeps (Sections 4.8, 4.9).

Hybrid evaluation protocol. Table 3 reports the primary Hybrid path (router $\rightarrow$ probe $\rightarrow$ reconstruct). For each historical run, we replay only the pre-run context $\mathbf{x}$ and the probe telemetry $\mathbf{p}$ from prior micro-probes on that run; the router, probe estimators, and reconstructor then produce $\hat{\mathbf{y}}$ and $\hat{Y}$. The reconstructor input contains *probe-imputed* scores $\hat{y}_j$ on the router-selected subset—never measured partial subtest scores from the full-run JSON. We compare $\hat{Y}$ against the reference full-suite index Y from the same run solely to compute error. This offline protocol avoids re-executing thousand-second full suites during cross-validation; Section 4.3 corroborates wall-time savings with online traces. *Router only* ablations (Sections 4.4–4.7) deliberately substitute *measured* partial scores to isolate routing and reconstruction; their higher errors (0.1–2%) must not be conflated with Hybrid headline numbers.

Metrics and Evaluation Criteria. We evaluate our methodology and summarize the results across testbeds using the following key metrics and aggregation strategies:

1. **Accuracy:** For the suite-level score $\hat{S}$ and reference full-run index S , we report absolute error and relative error. In the Hybrid path, $\hat{S}$ is produced entirely by model inference; S is used only as the evaluation label, never as a partial observation:

$$|\hat{S} - S|, \quad \frac{|\hat{S} - S|}{\max(|S|, \epsilon)}. \quad (15)$$

Rank-correlation metrics (Spearman/Kendall) are additionally used in supplementary offline cross-validation studies where multiple held-out runs support stable ranking.

2. **Cost:** We measure the wall-clock time of benchmark execution. Let T_{partial} be the executed subset duration

and T_{full} be the complete suite duration (or its dataset-replay equivalent). The fraction of time saved is

$$f_{\text{saved}} = \frac{T_{\text{full}} - T_{\text{partial}}}{T_{\text{full}}}. \quad (16)$$

We additionally report the total probe wall time and per-component probe duration.

3. **Combined trade-off score:** To evaluate Top- K configurations, we utilize a balanced score (where lower is better) that prioritizes prediction accuracy while treating wall-time savings as a secondary factor. Let e denote the mean out-of-fold suite relative error (Equation 15), $e_{\min} = \min e$ over the compared settings, and f_{saved} the mean fraction of full-suite wall time saved (Equation 16). We define the score as:

$$\text{Bal} = \frac{e}{e_{\min}} + \lambda (1 - f_{\text{saved}}), \quad (17)$$

where $\lambda = 0.15$. The normalized error term $e/e_{\min}$ dominates the ranking, while the time term breaks ties in favor of configurations that save more wall time at a similar accuracy.

4. **Feature-group importance:** In our system-feature ablation studies, let e_m denote the mean out-of-fold suite relative error under ablation mode m , and e_{full} the error using the complete feature vector. We measure feature importance by the *relative-error increase* when a group is removed:

$$\Delta e_m = (e_m - e_{\text{full}}) \times 100 \text{ pp}. \quad (18)$$

A larger Δe_m indicates that removing the feature group degrades prediction accuracy more severely, highlighting its importance. Ranks 1–4 order the ablated modes by descending Δe_m ; negative values indicate the removed group added noise or provided no benefit.

4.2 Overall Effectiveness

This is the primary BENCHSCOUT experiment: router Top- K , probe-model prediction on the selected subset, and suite reconstruction (Hybrid protocol above). We evaluate all $3 \times 2 \times 3$ router–probe–reconstructor combinations per host–suite pair and report the best mean suite relative error averaged over all collected historical runs on that host. Table 3 summarizes suite-level relative error and estimated partial wall time per host. UnixBench and PTS-CPU rows cover five hosts; PTS-GPU rows report H1 only. Every partial observation fed to the reconstructor is a probe-estimator output $\hat{y}_j$, not a measured subtest score from the reference full run. BENCHSCOUT achieves mean relative errors of $3.3 \times 10^{-5}\%$ (UnixBench), $3.4 \times 10^{-5}\%$ (PTS-CPU), and $2.2 \times 10^{-5}\%$ (PTS-GPU) while reducing wall time by 95–100% versus full-suite execution. These values reflect the full router–probe–reconstruct stack under probe-imputed partial observations; they are not *router only* results that substitute ground-truth partial scores. Execution-free MLP baselines incur the lowest

Table 3. Overall effectiveness across five heterogeneous hosts. BENCHSCOUT rows use the Hybrid pipeline: router Top-K, *probe-model-predicted* partial scores, and reconstruction (Section 4.2). Wang/Tousi rows are execution-free MLP proxies [? ?] (static hardware features only). Execution time is in seconds; lower is better for error and time. Bold marks best among tested methods.

Suite	Method	M1 (32U128G)		M2 (2U8G)		M3 (4U8G)		M4 (4U16G)		M5 (8U8G)	
		Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)
UnixBench	Full-suite	–	1691.0	–	1520.0	–	1580.0	–	1610.0	–	1550.0
	Wang (MLP)	0.21	3.5	2.67	3.5	6.28	3.5	4.47	3.5	4.37	3.5
	Tousi (MLP)	0.17	3.5	99.56	3.5	99.63	3.5	99.60	3.5	99.59	3.5
	BENCHSCOUT	3.3×10^{-5}	18.0	2.9×10^{-5}	18.0	3.1×10^{-5}	18.0	3.5×10^{-5}	18.0	3.2×10^{-5}	18.0
PTS-CPU	Full-suite	–	15755.0	–	12450.0	–	13100.0	–	13900.0	–	12800.0
	Wang (MLP)	85.54	3.5	99.51	3.5	99.66	3.5	99.99	3.5	99.79	3.5
	Tousi (MLP)	99.97	3.5	99.9990	3.5	99.9993	3.5	99.99997	3.5	99.9995	3.5
	BENCHSCOUT	3.4×10^{-5}	18.0	3.1×10^{-5}	18.0	3.6×10^{-5}	18.0	3.2×10^{-5}	18.0	3.3×10^{-5}	18.0
PTS-GPU	Full-suite	–	27018.0	–	–	–	–	–	–	–	–
	Wang (MLP)	0.26	3.5	–	–	–	–	–	–	–	–
	Tousi (MLP)	98.85	3.5	–	–	–	–	–	–	–	–
	BENCHSCOUT	2.2×10^{-5}	18.0	–	–	–	–	–	–	–	–

partial cost but exhibit high prediction error on PTS-CPU and most UnixBench hosts.

4.3 Runtime Overhead Analysis

Table 3 reports aggregate wall-time savings, but not how CPU and memory evolve during execution. We therefore capture online resource traces (0.5 s sampling, $t=0$ at run start) on three representative hosts—H1 (32U128G), H3 (4U8G), and H4 (4U16G)—under four UnixBench conditions: *Full run* (complete *perl Run -c 1*), *router only* (Top-K partial execution), *probe only* (twelve $D=4$ s micro-probes), and *BenchScout* (router, Top-K probes, and reconstruction in the traced window; x collection precedes the trace). Table 4 summarizes traced wall time and time-averaged CPU/memory utilization; Figure 5 overlays the corresponding waveforms on a shared ≈ 1680 s axis.

BenchScout finishes in ≈ 12 s on all three hosts— $\approx 99.3\%$ less wall time than a full run and $\approx 4\times$ faster than probe only—with the lowest CPU mean on H1 (4.0%) and no sustained memory penalty on H3/H4. Together, Table 4 and Figure 5 confirm that BENCHSCOUT incurs negligible runtime overhead: resource usage concentrates in the first minute rather than occupying the machine for the full UnixBench duration.

4.4 Model Combination Study

The *router only* path isolates the router–reconstructor stack when partial observations are *measured* sub-benchmark scores rather than probe predictions. We exhaustively evaluate all combinations of three routers (LightGBM, MLP, GNN-expert) and three reconstructors (XGBoost, LightGBM, MLP)—nine combinations per suite on H1—under online partial execution plus reconstruction (the *router only* grid protocol). Table 5 reports suite relative error (%) and benchmark wall-time

saved (%) for each combination; UnixBench uses the H1 grid, with matching H1 grids for PTS-CPU and PTS-GPU. Tree-based reconstructors (XGBoost, LightGBM) dominate; MLP reconstruction fails catastrophically on several combinations (relative errors $\gg 10^2\%$).

We identify MLP + LightGBM reconstruction for UnixBench (0.133% error) and PTS-CPU (0.218%), and LightGBM + LightGBM for PTS-GPU (1.60%), as the best tree-based *router only* combinations on H1. The *router only* supplementary studies in Sections 4.6 and 4.7 reuse these best pairs on H1; *probe only* evaluation (Section 4.5) uses independently trained probe regressors.

4.5 Probe Model Evaluation

This experiment evaluates whether short, time-bounded probes can provide sufficient signal for suite-level performance prediction. Each component is executed only in micro-probe mode for a fixed budget of 4 s, optionally collecting eBPF features, and the resulting probe features $\mathbf{p}_{i,j}$ are used to predict component scores. Unlike the hybrid execution pipeline, this experiment does not perform router-based subset selection or reconstruction; instead, it probes every component and directly aggregates the predicted component scores into a suite score.

We train two probe regressors, LightGBM and XGBoost, using probe features collected from each host’s historical full runs, and evaluate online prediction against the latest full run on the same host. For PTS suites, labels use $\log(1+x)$ scaling with per-profile estimators to handle heterogeneous metric magnitudes. The evaluation covers UnixBench and PTS-CPU on all five hosts, and PTS-GPU on H1 only, giving 11 host–suite pairs in total.

Table 4. UnixBench runtime resource summary on three representative hosts (0.5 s online sampling). Wall time is the end-to-end traced duration; CPU and memory columns report time-averaged system utilization over each trace. **Wall saved** is the reduction in traced wall time relative to a full-suite run on the same host. Bold highlights BenchScout (BENCHSCOUT).

Method	Wall time (s)			CPU mean (%)			Memory mean (%)			Wall saved (%)
	H1	H3	H4	H1	H3	H4	H1	H3	H4	
Full run	1702	1681	1679	15.6	35.9	35.5	21.9	17.2	8.6	–
Router only	387	377	377	33.5	23.4	22.5	38.0	18.6	9.0	77–78
Probe only	49	48	48	4.4	30.1	27.5	21.0	19.7	9.2	97
BenchScout	12	12	12	4.0	29.7	27.3	21.4	19.8	12.1	99.3

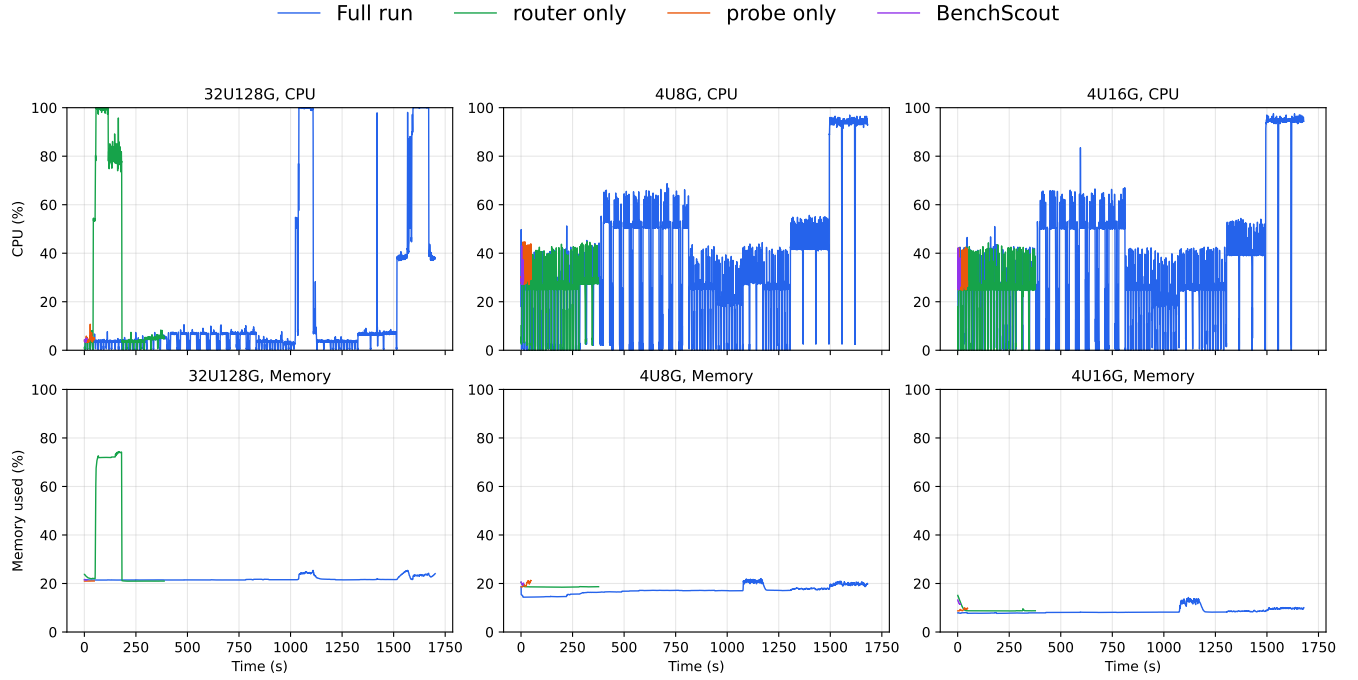


Figure 5. UnixBench CPU and memory waveforms on H1, H3, and H4 (0.5 s sampling). Rows: CPU utilization (%) and memory used (%); columns: 32U128G, 4U8G, and 4U16G. Curves show *Full run*, *router only*, *probe only*, and *BenchScout* on shared axes (≈ 1680 s horizon).

Table 6 compares LightGBM and XGBoost on H1 across all three suites. It reports suite relative error, probe wall time T_{partial} , full-suite wall time T_{full} from the ground-truth run JSON, and the resulting wall-time savings. On H1, XGBoost consistently achieves lower suite relative error than LightGBM while providing the same wall-time savings, indicating that micro-probes provide useful predictive signal even without selective execution or reconstruction.

4.6 Top- K Sweep

Using the best *router only* router–reconstructor pairs on H1 from Section 4.4 (MLP + LightGBM for UnixBench and PTS-CPU; LightGBM + LightGBM for PTS-GPU), we sweep the subset size K from 1 to 6, capped by the number of

components in each suite. For each K , the router selects Top- K experts, partial results are observed, and the reconstructor predicts the suite score. Evaluation uses leave-one-run-out cross-validation on H1’s historical runs in the canonical session per suite (random-fold when only one session exists), with router checkpoints and LightGBM reconstructors taken from each session’s *trained_models* directory; at each K , the reconstructor is trained with partial subsets of the same size as K .

Figure 6 visualizes the resulting error–saving trade-off, where each point corresponds to one candidate K . Although the figure does not plot the balanced score directly, the selected K is determined by minimizing Equation 17, which balances suite relative error against wall-time savings. On H1, the balanced score selects $K = 3$ for UnixBench, $K = 2$ for

Table 5. Router–reconstructor combination study on H1 with online partial execution and reconstruction.

Router	Reconstructor					
	XGB		LGBM		MLP	
	Err.	Save	Err.	Save	Err.	Save
<i>UnixBench</i>						
LightGBM	0.926	72.5	0.619	76.3	43.8	76.5
MLP	0.344	78.2	0.133	72.1	42.8	79.1
GNN-expert	0.282	76.5	0.415	76.5	59.0	76.5
<i>PTS-CPU</i>						
LightGBM	0.350	74.7	7.15	60.6	1.54×10^4	80.2
MLP	1.30	87.5	0.218	89.6	2.78×10^3	92.3
GNN-expert	3.12	95.5	3.14	39.6	1.65×10^4	79.5
<i>PTS-GPU</i>						
LightGBM	2.03	78.0	1.60	78.5	3.49×10^2	78.5
MLP	1.61	95.3	1.61	95.3	2.83×10^2	95.3
GNN-expert	1.88	95.4	1.60	94.9	4.03×10^2	95.3

Table 6. Probe model comparison on H1.

Suite	Model	Rel. err. (%)	T_{partial} (s)	T_{full} (s)	Saved (%)
UnixBench	LightGBM	0.66	48.0	319.9	85.0
	XGBoost	0.45	48.0	319.9	85.0
PTS-CPU	LightGBM	2.40	52.0	13018	99.6
	XGBoost	1.03	52.0	13018	99.6
PTS-GPU	LightGBM	2.60	40.0	27018	99.9
	XGBoost	2.28	40.0	27018	99.9

PTS-CPU, and $K = 3$ for PTS-GPU. The minimum balanced scores are 1.03 for UnixBench at $K = 3$, 1.01 for PTS-CPU at $K = 2$, and 1.01 for PTS-GPU at $K = 3$. These choices reflect that suite relative error decreases only modestly as K grows, while executing additional components quickly reduces wall-time savings. PTS-GPU includes only five GPU-compute profiles in our dataset, so the sweep is capped at $K = 5$.

4.7 System Context Feature Ablation

To quantify which parts of the system context $\mathbf{x}$ drive reconstruction accuracy, we ablate feature groups while holding router Top- K selection at $K=3$ on H1 (leave-one-run-out CV on each suite’s canonical session, random-fold fallback), using the same LightGBM reconstructor and *trained_models* checkpoints as in Section 4.6. We compare four ablated modes against a complete- $\mathbf{x}$ baseline (not shown): *static_hw_only* (hardware topology, memory, cache, and GPU inventory; zero dynamic/PMU), *no_perf_pmu* (remove PMU-derived rates except *perf_event Paranoid*), *no_dynamic_proc* (remove

images/topk_pareto_tradeoff.pdf

Figure 6. Top- K error–saving trade-off using the best router–reconstructor pair for each suite. Each point corresponds to a candidate K , with numbers indicating K . Stars mark the selected K according to the balanced score in Equation 17. Points closer to the lower-right corner achieve lower prediction error and higher time savings.

load, fault, and bandwidth-proxy dynamics), and *no_gpu* (remove GPU/OpenCL dimensions). Table 7 reports mean out-of-fold suite relative error and Δe_m (Equation 18) relative to the complete- $\mathbf{x}$ baseline. Importance rank (1 = most important to keep among the four ablated groups) sorts ablated modes by descending Δe_m . On PTS-CPU, ablating static hardware or dynamic /proc proxies increases error by ≈ 0.28 pp; PMU and GPU groups have smaller effects. On UnixBench all Δe_m magnitudes stay below 0.01 pp on H1. On PTS-GPU, *no_gpu* yields the largest shift ($\Delta e_m = -0.10$ pp), indicating GPU/OpenCL inventory features most strongly condition reconstruction among the ablated groups.

4.8 Probe Telemetry Degradation (eBPF On vs. Off)

Section 3.4 and Table 1 describe optional eBPF-derived scheduling and syscall rates as part of the in-run probe vector $\mathbf{p}_{i,j}$. In restricted environments, BENCHSCOUT must not abort when bpftrace is unavailable; instead it sets the *ebpf_available* flag to zero and zero-masks the corresponding probe dimensions. We validate this *graceful degradation* on H1 (32U128G) with a controlled *Probe only* experiment on UnixBench.

Protocol. We fix the trained H1 LightGBM probe regressor (micro mode, 4 s per component) and run two online conditions back-to-back on the same host: (i) **eBPF on**—bpftrace enabled (with elevated privileges); and (ii) **eBPF off**—tracing

Table 7. System-feature ablation at fixed router policy and $K = 3$ (offline CV on H1). Δe = relative-error change vs. complete $\mathbf{x}$ baseline (Equation 18; pp = percentage points; baseline row omitted). Importance rank orders ablated modes by descending Δe .

Suite	Ablation mode	Rel. err. (%)	Δe (pp)	Imp. rank
UnixBench	<i>static_hw_only</i>	0.241	−0.003	2
	<i>no_perf_pmu</i>	0.243	0.000	1
	<i>no_dynamic_proc</i>	0.241	−0.003	3
	<i>no_gpu</i>	0.237	−0.007	4
PTS-CPU	<i>static_hw_only</i>	2.04	+0.28	1
	<i>no_perf_pmu</i>	1.76	0.00	3
	<i>no_dynamic_proc</i>	2.04	+0.28	2
	<i>no_gpu</i>	1.75	−0.01	4
PTS-GPU	<i>static_hw_only</i>	0.957	+0.04	1
	<i>no_perf_pmu</i>	0.918	0.00	3
	<i>no_dynamic_proc</i>	0.957	+0.04	2
	<i>no_gpu</i>	0.818	−0.10	4

Table 8. eBPF degradation on H1 UnixBench *Probe only* (LightGBM; $D=4$ s; $Y=3185.5$). **sched**: mean context-switch rate ($10^4/s$).

Telem.	Err (%)	$ \hat{Y}-Y $	$\hat{Y}$	T_{pr} (s)	eBPF ok	sched
On	0.27	8.48	3194.0	61.8	12/12	8.3
Off	0.17	5.42	3190.9	60.6	0/12	—

disabled, with eBPF dimensions zero-masked. Each condition probes all 12 UnixBench components, predicts subtest indices, aggregates the suite score, and compares against ground truth from the latest full run in the canonical session ($Y=3185.5$). The probe model is trained on H1 historical runs using the same micro-probe protocol and eBPF-enabled feature collection.

Results. Table 8 summarizes accuracy, predicted suite score, probe cost, and eBPF availability; both conditions finish all 12 probes without error. Suite relative error remains sub-percent in either mode (0.27% with eBPF on vs. 0.17% with eBPF off), with nearly identical probe wall time (≈ 62 s vs. ≈ 61 s) and comparable $|\hat{Y} - Y|$ (8.5 vs. 5.4 index points). Under eBPF off, the fallback path still yields a stable suite estimate ($\hat{Y}=3190.9$). These results confirm that BENCHSCOUT tolerates disabled eBPF telemetry without aborting evaluation and without large accuracy loss on this host.

4.9 Probe Duration Sensitivity

Section 3.4 introduces a fixed per-component probe budget as the primary knob for trading *Probe only* latency against predictive fidelity. While the default configuration uses 4 s micro-probes, practitioners may tighten this budget in latency-sensitive workflows (e.g., autotuning loops or co-located monitoring) or relax it when sub-percent accuracy is paramount. We quantify this accuracy–cost frontier on H1 (32U128G) with a controlled UnixBench *Probe only* sweep over probe durations $D \in \{2, 4, 8\}$ s.

Protocol. For each duration D , we run an independent end-to-end pipeline so that training and inference observe the same time budget: **(i) dataset collection**—replay labels from H1’s canonical UnixBench session and collect fresh micro-probes at duration D for all 12 components (60 labeled rows total); **(ii) regressor training**—fit a dedicated LightGBM probe model on the duration- D dataset (shared estimator with test-id one-hot encoding, eBPF disabled to isolate duration effects); **(iii) online evaluation**—probe every component once at duration D , predict subtest indices, aggregate the suite score, and compare against ground truth from the latest full run ($Y=3185.5$). Each condition completes all 12 probes without timeout or pipeline failure. We report suite relative error, total probe wall time T_{partial} , and wall-time savings relative to the ground-truth full-suite duration ($T_{\text{full}}=319.9$ s from the same run JSON).

Results. Table 9 summarizes the sweep. Suite relative error decreases monotonically with longer probes—from 0.80% at $D=2$ s to 0.56% at $D=4$ s and 0.44% at $D=8$ s—while T_{probe} grows from 36.4 s to 60.4 s and 108.6 s ($\approx 1.7\times$ and $\approx 3.0\times$ the 2 s cost, respectively) and $|\hat{Y} - Y|$ shrinks from 25.4 to 13.9 index points. The predicted suite scores ($\hat{Y}=3210.9$, 3203.3, and 3199.4) all lie within 26 index points of ground truth, confirming that even aggressive 2 s micro-probes retain sub-percent fidelity on this host; all conditions complete 12/12 probes. The marginal accuracy gain from doubling the budget (2 s $\rightarrow$ 4 s: -0.24 pp; 4 s $\rightarrow$ 8 s: -0.12 pp) diminishes as D increases, whereas wall-time cost scales nearly linearly with the nominal $D \times 12$ component budget plus fixed per-probe overhead (≈ 12 s total across all durations). For latency-constrained *Probe only* deployments, $D=2$ s offers the best accuracy–cost point in this sweep (88.6% wall-time saved, $<1\%$ error); $D=4$ s remains a balanced default consistent with the rest of our H1 probe experiments.

5 Related Work

5.1 System Benchmarking and Performance Evaluation

Benchmarking is a widely used methodology for evaluating the performance of operating systems [?]. A major line of OS benchmarking focuses on measuring low-level operating-system primitives, such as system-call overhead, context

Table 9. Probe duration sweep on H1 UnixBench *Probe only* (LightGBM; micro; eBPF off; $Y=3185.5$, $T_{\text{full}}=319.9$ s).

D (s)	Err(%)	$ \hat{Y}-Y $	$\hat{Y}$	T_{pr} (s)	Saved(%)	Probe-ok
2	0.80	25.4	3210.9	36.4	88.6	12/12
4	0.56	17.8	3203.3	60.4	81.1	12/12
8	0.44	13.9	3199.4	108.6	66.1	12/12

switching, process creation, interprocess communication, memory latency, cache behavior, file operations, and network performance. Representative microbenchmark suites include LMBench [?], UnixBench [?], and HBBench-OS [?]. Brown and Seltzer [?] extended the LMBench methodology and introduced HBBench-OS to improve benchmarking flexibility, precision, and statistical rigor. Beyond performance, Koopman et al. [?] proposed a robustness benchmarking method for operating systems using response regions and a CRASH severity scale to characterize erroneous system behavior. Kanoun et al. [?] summarized principles and examples of dependability benchmarking, emphasizing reliability, availability, serviceability, and robustness metrics for computer-system evaluation. OS benchmarking has also been adapted to domain-specific operating systems, where evaluation metrics are determined by deployment requirements. Garre et al. [?] compared RTOS and GPOS platforms for complex real-time simulations and showed that RTOS platforms can be preferable due to lower IPC and task-switching latency under parallel simulation workloads. Watfa et al. [?] proposed a benchmarking and performance evaluation approach for embedded operating systems in wireless sensor networks, highlighting the tradeoff between event-driven and thread-driven designs. Silva et al. [?] benchmarked IoT operating systems such as Contiki-NG, RIOT, and Zephyr using criteria including low-power consumption, real-time capability, security, interoperability, and connectivity.

5.2 Performance Prediction

Simulation-based and statistical modeling methods reduce architectural evaluation cost by simulating only selected configurations and using the results to estimate unexplored design points [? ? ? ?]. For example, Ipek et al. [?] learned predictive models from sampled architectural simulations to explore large design spaces, while Lee and Brooks [?] used regression models to predict performance and power from a tiny fraction of simulated microarchitectural configurations. Analytical and mechanistic models estimate performance by explicitly modeling processor behavior, such as pipeline effects, cache misses, prefetching, MSHR resources, and execution intervals [? ? ? ? ?]. For instance, Eyerman et al. [?] modeled out-of-order execution as intervals separated by disruptive miss events, and Chen and Aamodt [?] modeled the performance impact of pending cache hits, prefetching, and limited MSHR resources.

Machine-learning-based methods have also been widely used to learn complex relationships among programs, hardware characteristics, and performance outcomes. Ardalani et al. [?] proposed XAPP, which predicts GPU execution time from features of a single-threaded CPU implementation before GPU porting. Zheng et al. [?] introduced LACross, which predicts phase-level cross-platform performance and power traces from hardware-counter statistics collected on a host platform. Wang et al. [?] used deep neural networks to predict benchmark performance for new CPUs and workloads from public CPU specifications and benchmark datasets.

6 Conclusion

Full-suite system benchmarking remains the gold standard for comparing platforms, yet its all-or-nothing execution model makes repeated measurement impractical in autotuning, multi-machine regression, and production-adjacent monitoring. We presented BENCHSCOUT, which reframes suite evaluation as Top- K routing, time-bounded micro-probes, and sparse reconstruction of full suite profiles from partial observations. On five heterogeneous Linux hosts, the Hybrid path recovers UnixBench and PTS scores with near-zero relative error while cutting wall time by 95–100% versus full-suite execution (Section 4), showing that standard benchmark indices can be approximated with seconds of bounded on-host observation rather than tens of minutes of full runs.

7 Limitation and Future Work

Our evaluation trains and validates models per host; cross-host transfer and cold-start routing with limited historical data remain open challenges. Results rely on offline replay of stored system context and probe telemetry rather than fully live end-to-end sessions; extending validation to additional suites, cloud SKUs, and one-shot collection-inference pipelines would strengthen deployment claims. Live integration into autotuning loops—with adaptive probe budgets and K —is a natural next step toward routine, low-disruption benchmarking in systems research and operations.

References

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009